

Tecnologia em Jogos Digitais • Inteligência Artificial e Ilusão de Inteligência

Inteligência Artificial e Ilusão de Inteligência

Semana 3 — Aprofundamento da FSM: State por herança em C#

Módulo 1: Como um NPC decide o que fazer?

Apostila: Parte II, Cap. 3 (aprofundamento)

Micro Game: NPC Decision

IFMS • Semana 03

Objetivos da aula

Ao final de hoje você será capaz de:

- explicar o problema de um `switch` de FSM que cresce demais;
- descrever a estrutura do padrão State por herança (classe base + métodos virtuais);
- justificar por que herança se encaixa melhor aqui do que interface;
- mapear estado, transição, evento e ação para essa estrutura;
- refatorar a FSM da Semana 2 usando uma classe base `State`.

Antes de começar: mudança de rota

⚠️ ATENÇÃO

Esta turma **não vai estudar Máquinas de Estado Hierárquicas** (Capítulo 4) neste semestre. O Micro Game segue direto de FSM para Árvore de Comportamento, na Semana 4.

Hoje o assunto não é "como organizar hierarquicamente" — é "como organizar o **código**" da FSM que você já tem.

Retomada da Semana 2

Você já tem uma FSM funcional: estados, transições, eventos e ações, implementados por `enum` / `switch` (ou já orientada a objetos, para quem escolheu esse caminho).

O que acontece com esse código quando o número de estados cresce?



Como estruturar o código da FSM para que ela cresça sem virar um emaranhado de condicionais?

O guarda tem cinco estados. Cada um com sua própria lógica de update. E tudo mora no mesmo `switch`.

O problema: um método que cresce demais

Não é a quantidade de estados — é que **toda a lógica de todos os estados convive no mesmo lugar**.

⚠️ ATENÇÃO

Um `switch` com cinco `case`, cada um com dezenas de linhas, é difícil de ler, difícil de testar isoladamente e fácil de quebrar ao editar um caso vizinho.

A resposta: o padrão State — por herança

Cada estado vira uma **classe própria**, que **herda** de uma classe base comum.

- Uma classe base `State`, com métodos **virtuais** (`Enter`, `Update`, `Exit`, `FixedUpdate`)
- Uma subclasse por estado (`PatrulharState`, `AtacarState` ...), sobrescrevendo só o que precisa
- Um **contexto** — a `FSM` — que mantém o estado ativo e delega as chamadas

Código real: State.cs

```
public class State
{
    protected FSM _fsm;
    public State(FSM fsm) { _fsm = fsm; }

    public virtual void Enter() { }
    public virtual void Exit() { }
    public virtual void Update() { }
    public virtual void FixedUpdate() { }
}
```



DICA

Métodos `virtual` com corpo vazio: cada subclasse sobrescreve só o que precisa.

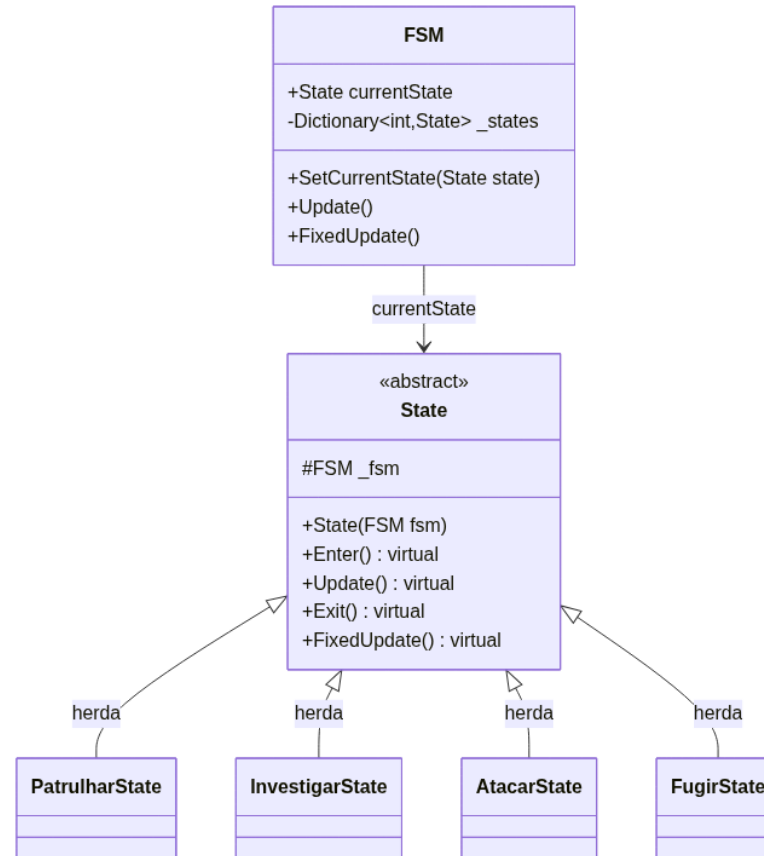
Código real: FSM.cs

```
public class FSM
{
    public State currentState;
    Dictionary<int, State> _states = new Dictionary<int, State>();

    public void SetCurrentState(State state)
    {
        currentState?.Exit();
        currentState = state;
        currentState?.Enter();
    }

    public void Update() => currentState?.Update();
}
```

Estrutura do padrão State por herança



Por que herança, e não interface?

Herança (State base)

Compartilha o campo `_fsm` pronto para todo estado

Métodos virtuais: sobrescreve só o que precisa

Estado não pode herdar de outra classe além de `State`

Interface (IState)

Cada classe reimplementaria seu próprio campo de contexto

Obriga implementar os quatro métodos, mesmo vazios

Uma classe pode implementar várias interfaces

DICA

Aqui os estados são objetos simples (não `MonoBehaviour`), então a limitação de herança única não pesa — a herança compensa pelo campo e pelos métodos compartilhados.



Erro de projeto no código real

`State` não é `abstract`. Nada impede `new State(fsm)` — um "estado genérico" sem comportamento algum.

OBJETIVOS

Pergunta-chave: um estado sem lógica faz sentido existir? Se não, a classe deveria ser `public abstract class State`.

Mapeando a Semana 2 para a classe State

Conceito (Semana 2)	Elemento na classe State / FSM
Estado	Subclasse de State
Ciclo enter/update/exit	Métodos virtuais sobrescritos (+ FixedUpdate , para física)
Transição	Chamada a fsm.SetCurrentState(novoEstado)
Evento/guarda	Condição avaliada dentro do Update / FixedUpdate do estado ativo

Exemplo: do switch à subclasse

```
// Antes: um case dentro de um switch gigante
case EstadoGuarda.Atacar:
    if (vida < 0.2f) TrocarPara(EstadoGuarda.Fugir);
    AtacarJogador();
    break;
```

```
// Depois: uma subclasse isolada
public class AtacarState : State
{
    public AtacarState(FSM fsm) : base(fsm) { }
    public override void Update()
    {
        if (guarda.Vida < 0.2f) _fsm.SetCurrentState(new FugirState(_fsm));
        guarda.AtacarJogador();
    }
}
```

O que a refatoração melhora

- **Isolamento:** a lógica de "Atacar" só existe em `AtacarState`
- **Legibilidade:** cada arquivo tem uma responsabilidade só
- **Testabilidade:** dá para testar um estado sem montar a FSM inteira
- **Menos boilerplate:** só sobrescreve o que o estado realmente usa



O que a refatoração não resolve

OBJETIVOS

O número de regras de transição continua o mesmo — só está mais organizado. A explosão de transições, identificada na Semana 2, segue de pé.

Tecnologia em Jogos Digitais • Inteligência Artificial e Ilusão de Inteligência

Implementação guiada

Refatorando a FSM da Semana 2 para herança de `State`.

IFMS • Semana 03

Passo a passo

1. Revisar como sua FSM foi implementada (Animator, `switch` ou já orientada a objetos);
2. Definir a classe base `State` (**abstrata**), com `_fsm` e os métodos virtuais;
3. Criar uma subclasse para cada estado já existente;
4. Implementar o contexto `FSM`, que troca entre estados via `SetCurrentState`;
5. Testar: o comportamento deve ser **idêntico** ao de antes.

O contexto: quem manda na troca

DICA

O estado decide **quando** pedir a troca (chamando `_fsm.SetCurrentState(...)` dentro do seu `Update`). O contexto (`FSM`) é quem de fato **realiza** a troca — disparando `Exit` do antigo e `Enter` do novo — e guarda a referência ao estado ativo.



Erro comum

Colocar toda a lógica de transição dentro de cada subclasse de estado, sem consultar o contexto central que controla qual estado está ativo.

OBJETIVOS

Pergunta-chave: "quem guarda a referência ao estado ativo?" Se a resposta não for "o contexto (FSM)", reveja o desenho.

Discussão técnica: `switch` versus herança de `State`

Compare a legibilidade e a testabilidade do código antes e depois da refatoração.

⚠ ATENÇÃO

A quantidade de transições escritas mudou? Não. O que ainda falta resolver é a mesma coisa de sempre: reordenar prioridades e reutilizar comportamento continuam trabalhosos — gancho para a Semana 4.

Resumo da semana

- O `switch` de FSM cresce mal quando o número de estados aumenta
- O padrão State por herança isola cada estado em uma subclasse, com métodos virtuais sobrescritos sob demanda
- Herança compartilha o campo de contexto e evita boilerplate — a troca por interface exigiria implementar tudo em cada classe
- Estado, transição, evento e ação mapeiam diretamente para `State` / `FSM`
- A explosão de transições **continua sem solução** — não é isso que a refatoração resolve

Preparação para a Semana 4

Tema: Árvores de Comportamento, Blackboard e Engenharia Reversa

- Ler o Capítulo 6 da Apostila (Árvores de Comportamento) e o Capítulo 14 (Engenharia Reversa)
- Trazer a FSM (herança de `State`) funcional implementada hoje, no Micro Game NPC Decision



DICA

Pergunta que abre a Semana 4: como tornar a decisão do NPC modular e escalável, migrando direto da FSM para uma Árvore de Comportamento?